

CynLr - Chasing the Dot

Machine Learning / Reinforcement Learning Assignment
Sri Ramvarma Yembadi
30 September 2026

Submission scope: empirical system investigation first; explicit assumption cases second. A concrete RL model is not claimed as a validated description of the runtime.

1. Summary

The task is to control a Blue Dot while a Green Dot follows a path. I first implemented the TCP protocol/client and then treated the application as a black box before choosing an RL algorithm. I established the live connection, packet structure, big-endian interpretation, continuous observations, and that P commands affect Blue-related behaviour. I also found timing-dependent behaviour and could not safely establish a causal mapping between an individual command and an individual observation.

Because those unknowns affect the definition of the state, transition, reward clock and timestep, I separate the empirical findings from the assumptions used to discuss how an RL model could be built later. The detailed theoretical treatment is kept in the repository documentation.

2. Problem and application interface

Interface element	Working interpretation
Observation	20-byte record: Green X/Y, Blue X/Y, Error X/Y, CR, LF
P command	13-byte command containing an absolute Blue X/Y coordinate
C command	17-byte configuration command for Object Speed and Object Size
Transport	One TCP connection carrying observations and commands
Byte order	Big endian, verified from the live application

3. Approach: reduce uncertainty before RL

I used small experiments, each aimed at one uncertainty rather than immediately tuning an RL agent.

Experiment	Question
Passive observation stream	Does the application continuously produce observations without commands?
One P command	Does a single position command affect Blue behaviour?
Timestamped P	What happens in the first few tens of milliseconds after a command?
Repeated fixed P	Does maintaining a command behave differently from sending it once?
Command refresh timing	Does command frequency change observed behaviour?
Object Size effect	What changes when configuration is modified?
Simple Green tracker	Can a basic feedback controller follow the reported Green position?
Error metric probe	Can GUI Error % or Error X/Y be treated as a per-step learning signal?

4. Findings from the live investigation

Established: TCP works; framing/parsing work; big-endian interpretation matches the live protocol; observations stream continuously; P affects Blue-related behaviour; command timing/frequency changes what is observed; faster Green motion was qualitatively harder to follow with the simple tracker.

Not established: exact P temporal semantics; command-to-observation causality; exact delay/queue behaviour; Error X/Y semantics; GUI Error % as a per-step learning signal; exact coordinate bounds; whether the visible observation is Markov without history.

I stopped here because the remaining uncertainties change the environment definition itself. Choosing an algorithm first would have meant silently assuming away the hardest part.

5. Error metric investigation

I tested whether the GUI-level Error % and the two one-byte Error X/Y fields could provide a clean per-step learning signal. In the probe, the GUI Error % stayed at 100% before, during and after the follow/hold phases, while the raw flags remained asserted even in observations where the streamed Green and Blue coordinates were equal.

I therefore do not interpret those fields as a verified learning signal. Coordinate distance can still be used diagnostically when the streamed coordinates are known to share a common geometry, but I do not elevate that diagnostic quantity to an empirically validated reward.

6. Explicit assumption cases

The key point is that different assumptions lead to different problem classes. I keep the cases separate instead of assuming one hidden environment.

6.1 Case A - instantaneous Blue teleportation

- Blue and Green coordinates are true positions in a common coordinate system.
- $P(X,Y)$ immediately places Blue at that integer coordinate.
- Observation and action delay is negligible for the decision.
- Green is observed before the action.

Implication: direct feedback control may be sufficient, so RL may not be necessary in this boundary case.

6.2 Case B - fixed one-step delay with uncertain Green motion

- Blue still teleports, but the command chosen at step t takes effect at $t+1$.
- Green moves on an integer grid.
- Green's next direction is one of the four cardinal directions.
- The next direction is not assumed deterministic or inferable from a short history.
- Recent Green history is available.

Implication: with an additional Markov assumption, the environment can be treated as a discrete decision process on the coordinate grid. This is an assumption-dependent route to an RL model, not a validated model of the application.

6.3 Case C - hidden or variable command delay

- $P(X,Y)$ affects Blue, but the effective delay is hidden or variable.
- Observation records arrive asynchronously relative to commands.
- Internal command-processing state is not directly observed.

Implication: the visible process may be non-Markov. A history-aware or memory-based model could therefore be required if this is what the runtime actually does.

7. What I do not assume

- Green future motion is deterministic.
- A short history always reveals the future Green path.
- The testing trajectory is known in advance.
- The four-direction assumption by itself makes the process Markov.

The four-direction assumption defines only the set of possible next directions; it does not establish which direction will occur. Likewise, a predictable full future path is only a counterfactual model-based case, not an assumption of the testing task.

8. What these assumptions mean for a possible RL model

The purpose of the assumption cases is to identify what must be true before an RL formulation is technically meaningful. The cases do not, by themselves, justify training an agent on the current runtime.

Instantaneous teleportation: the control problem can collapse to direct feedback control.

Fixed one-step delay: a discrete-action RL formulation becomes possible once the Markov/state assumptions, legal coordinates, and reward clock are established.

Hidden or variable delay: the model may need history or recurrent memory because the visible observation can omit relevant internal state.

The detailed state/action/transition/reward alternatives are intentionally kept in **docs/theoretical_rl_assumptions.md**. This report records the assumptions and their implications without presenting one concrete RL formulation as the answer.

9. Discrete action space

The `P` command supplies integer X/Y coordinates. If the legal workspace is $x = 0, \dots, W$ and $y = 0, \dots, H$, the coordinate-pair action set is finite, with $|A| = (W+1)(H+1)$. The space may be large, but it is discrete rather than continuous. This is a property of the interface, not a claim about a trained policy.

The important remaining question is which runtime assumptions are actually true. The detailed alternatives are kept in **docs/theoretical_rl_assumptions.md**.

10. What would need to be established before empirical RL

- a reliable command-to-observation timing convention;
- practical legal coordinate bounds;
- a defined decision interval or an explicit variable-delay model;
- a verified learning/reward measurement clock;
- an episode reset/end convention;
- evidence that the chosen state is sufficiently Markov, or an explicit memory mechanism.

11. Conclusion

My main decision was to separate system identification from algorithm selection. The empirical work establishes a usable TCP interface and several important behaviours, but not a fully identified transition model.

The assumptions show that the same interface could correspond to direct control, a discrete decision process, or a partially observable control problem depending on the actual timing semantics. A concrete RL model can therefore be built once those semantics are established, but presenting a trained agent now would overstate what the experiments support.

This report deliberately stops at that boundary. The repository documentation contains the more detailed theoretical treatment for a later RL implementation.